CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

DEPARTAMENTO DE COMPUTAÇÃO (DECOM)

Engenharia de Computação

Laboratório de Organização e Arquitetura de Computadores I

PRÁTICA 7

Unidades de Controle (UC) e Caminho de Dados Completo das Instruções Lógicas e Aritméticas

Aluno: Vítor Albuquerque Dinelli

Professora: Poliana Corrêa

Belo Horizonte, outubro de 2026

1. Introdução

Nesta prática implementei, em Verilog, as duas unidades de controle do processador MIPS (controle principal e controle da ULA) e as integrei ao caminho de dados que já vinha sendo construído nas práticas anteriores. Com isso, o processador passa a buscar uma instrução na memória, decodificá-la, executá-la na ULA e gravar o resultado no banco de registradores, para as instruções do tipo R (add, sub, and, or, slt) e para o addi.

O controle principal olha apenas para o opcode (bits 31:26) e gera os sinais que comandam os multiplexadores, o banco de registradores e a memória. O controle da ULA combina o campo ALUOp, vindo do controle principal, com o campo funct (bits 5:0) para gerar os 3 bits que selecionam a operação da ULA. A validação foi feita por simulação no ModelSim/Questa, com testbenches individuais para os dois blocos de controle e testbenches autoverificáveis para o caminho de dados completo. O projeto também foi levado à placa DE10-Lite, com um top-level que permite acompanhar cada etapa da instrução nos displays e LEDs.

2. Projeto dos blocos

2.1 Controle principal (control.v)

Implementei o controle de forma comportamental, com o decodificador de opcode descrito por portas lógicas (assign), seguindo a lógica da Figura 2 do roteiro: um sinal por formato de instrução (R_format, lw, sw, beq) e as saídas obtidas por OR desses sinais. Como o roteiro pede o addi na demonstração e ele não consta na tabela original, acrescentei um quinto decodificador (opcode 001000). O addi usa o imediato como segundo operando (ALUSrc=1), escreve em rt (RegDst=0), grava no banco (RegWrite=1) e pede soma à ULA (ALUOp=00).

Os valores "don't care" (X) da tabela do roteiro (RegDst e MemtoReg para sw e beq) foram implementados como 0, o que não afeta o funcionamento, já que nessas instruções RegWrite=0.

Instrução	Op	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
Tipo R	000000	1	0	0	1	0	0	0	10
LW	100011	0	1	1	1	1	0	0	00
SW	101011	0	1	0	0	0	1	0	00
BEQ	000100	0	0	0	0	0	0	1	01
ADDI	001000	0	1	0	1	0	0	0	00

Tabela 1 – Saídas implementadas no controle principal

```
//so copiei as portas q tava no slides caminhos de dados do processador e adicio nei o addi pq
la n tinha
module control(
  input [5:0] Op,
  output RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch,
  output [1:0]ALUOp
);

  //definindo qual tipo de instruçao vai ser
  wire R_format = ~|Op;
  wire lw = Op[5] & ~Op[4] & ~Op[3] & ~Op[2] & Op[1] & Op[0];
  wire sw = Op[5] & ~Op[4] & Op[3] & ~Op[2] & Op[1] & Op[0];
```

```
  wire beq = ~Op[5] & ~Op[4] & ~Op[3] & Op[2] & ~Op[1] & ~Op[0];
  wire addi = ~Op[5] & ~Op[4] &  Op[3] & ~Op[2] & ~Op[1] & ~Op[0];

  //setandos os outputs
  assign RegDst = R_format;
  assign ALUSrc = lw | sw | addi;
  assign MemtoReg = lw;
  assign RegWrite = R_format | lw | addi;
  assign MemRead = lw;
  assign MemWrite = sw;
  assign Branch = beq;
  assign ALUOp[1] = R_format;
  assign ALUOp[0] = beq;

endmodule
```

2.2 Controle da ULA (ULActrl.v)

Este bloco recebe ALUOp (2 bits) e funct (6 bits) e gera operation (3 bits). Implementei as três equações da Figura 1 do roteiro diretamente com portas lógicas. Apenas os bits F3, F2, F1 e F0 do funct são necessários para distinguir as operações:

- operation[2] = ALUOp[0] | (ALUOp[1] & funct[1])
- operation[1] = ~ALUOp[1] | ~funct[2]
- operation[0] = ALUOp[1] & (funct[3] | funct[0])

Instrução	ALUOp	funct	Operação	Saída
LW / SW	00	XXXXXX	add	010
BEQ	01	XXXXXX	subtract	110
ADD	10	100000	add	010
SUB	10	100010	subtract	110
AND	10	100100	AND	000
OR	10	100101	OR	001
SLT	10	101010	set on less than	111

Tabela 2 – Entradas e saída do controle da ULA

```
//so copiei as portas q tava no slides caminhos de dados do processador
module ULActrl(
  input [5:0]funct,
  input [1:0]ALUOp,
  output [2:0]operation
);
  assign operation[2] = ALUOp[0] | (ALUOp[1] & funct[1]);
  assign operation[1] = ~ALUOp[1] | ~funct[2];
  assign operation[0] = ALUOp[1] & (funct[3] | funct[0]);
endmodule
```

2.3 Demais blocos do caminho de dados

- **ULA (ULA.v):** ULA de 32 bits já desenvolvida em práticas anteriores, sem alterações. Realiza and, or, soma, subtração e slt conforme os 3 bits de operation do controle da ULA, e gera a flag zero usada pelo beq.

- **PC (PC.v):** registrador de 32 bits com reset assíncrono (PC=0) e atualização na borda de subida do clock.
- **Somadores do PC:** dois parameterAdder de 32 bits; um calcula PC+4 e o outro PC+4 + (imediato deslocado 2 bits à esquerda), o endereço de desvio do beq. A escolha é feita por PCprox = (Branch & zero) ? PCbeq : PCmais4.
- **Memória de instruções (memoriaInstrucoes.v):** inicializada com $readmemb a partir do arquivo instrucoes.txt (uma instrução de 32 bits por linha). Por limitação de memória da FPGA, o endereço usa 8 bits (256 bytes) em vez de 32.
- **Banco de registradores (bancoDeReg.v):** 32 registradores de 32 bits, leitura assíncrona de duas portas e escrita síncrona na borda de subida quando RegWrite=1. É inicializado por $readmemh a partir de registradores.txt (valores em hexadecimal); usei $17=7 e $18=0x19 (25). A escrita é bloqueada durante o reset (RegWrite & ~reset).
- **Extensor de sinal (extensorSinal.v):** estende o imediato de 16 para 32 bits replicando o bit de sinal.
- **Multiplexadores:** WriteReg = RegDst ? instr[15:11] : instr[20:16] e ALUb = ALUSrc ? imediato : Data2.
- **Módulo cpuMimps.v:** interliga todos os blocos acima e expõe como saídas os sinais internos (PC, instrução, Data1, Data2, ALUb, imediato, resultado da ULA, WriteReg, sinais de controle), facilitando as simulações e o top-level. A memória de dados não foi incluída, conforme permitido no roteiro; os sinais MemRead, MemWrite e MemtoReg são gerados pelo controle, mas não têm destino no caminho implementado.

2.4 Top-level para a placa DE10-Lite (Top_cpu.v)

Para a demonstração, o clock do processador é manual: KEY[0] gera um pulso por clique (passando por um circuito de debounce de cerca de 21 ms) e KEY[1] é o reset. As chaves SW[3:0] escolhem o que aparece nos seis displays de 7 segmentos, de modo que cada passo da instrução possa ser visto separadamente. Os valores numéricos são convertidos de binário para BCD (bin2bcd, algoritmo double dabble) e mostrados em decimal, com sinal quando necessário.

SW[3:0]	Display	Etapa do caminho de dados
0	PC	Busca
1	Opcode (instr[31:26])	Decodificação
2	rs | rt | rd	Decodificação
3	funct (instr[5:0])	Decodificação
4	Imediato estendido	Decodificação
5	Data1	Leitura do banco
6	Data2	Leitura do banco
7	ALUb	Entrada B da ULA (após mux ALUSrc)
8	ALUResult	Execução / dado a gravar
9	WriteReg	Registrador destino (após mux RegDst)
10	ALUOp | operação da ULA	Controle da ULA

Tabela 3 – Seleção dos displays pelas chaves SW[3:0]

Os LEDs LEDR[0..6] mostram RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite

e Branch; LEDR[7] indica ALUResult = 0, LEDR[8] indica que o valor não cabe nos 5 dígitos e LEDR[9] acompanha o clock manual.

2.5 Programa executado

O programa pedido no item II.a foi traduzido para linguagem de máquina e armazenado em instrucoes.txt:

End.	Instrução	opcode	rs	rt	rd / imediato	shamt	funct
0	addi $9,$18,10	001000	10010	01001	0000000000001010	–	–
4	sub $9,$18,$17	000000	10010	10001	01001	00000	100010
8	and $8,$9,$18	000000	01001	10010	01000	00000	100100
12	or $10,$9,$17	000000	01001	10001	01010	00000	100101
16	slt $8,$18,$17	000000	10010	10001	01000	00000	101010

Tabela 4 – Programa em linguagem de máquina

Com $17=7 e $18=25, o resultado esperado de cada instrução é:

Instrução	Data1	ALUb	Operação	Resultado	Destino
addi	25	10	010	35	$9
sub	25	7	110	18	$9
and	18	25	000	16	$8
or	18	7	001	23	$10
slt	25	7	111	0	$8

Tabela 5 – Execução esperada do programa

3. Simulações

3.1 Controle da ULA (tb_ULActrl.v)

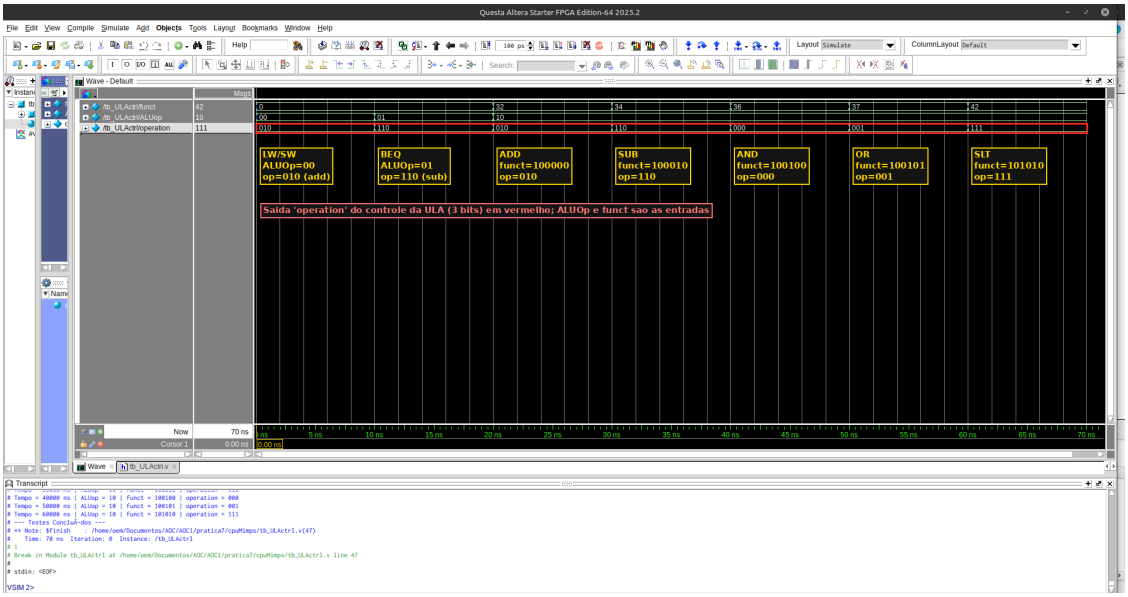


Figura 1 – Forma de onda do controle da ULA

O testbench aplica, a cada 10 ns, as combinações de ALUOp e funct da Tabela 2. Com ALUOp=00 (LW/SW) a saída é 010 (soma) e com ALUOp=01 (BEQ) é 110 (subtração), ambas

independentes do funct. Com ALUOp=10 a saída acompanha o funct: 32→010 (add), 34→110 (sub), 36→000 (and), 37→001 (or) e 42→111 (slt). A simulação evidencia que o bloco gera exatamente os códigos esperados pela ULA em todos os casos exigidos.

3.2 Controle principal (tb_control.v)

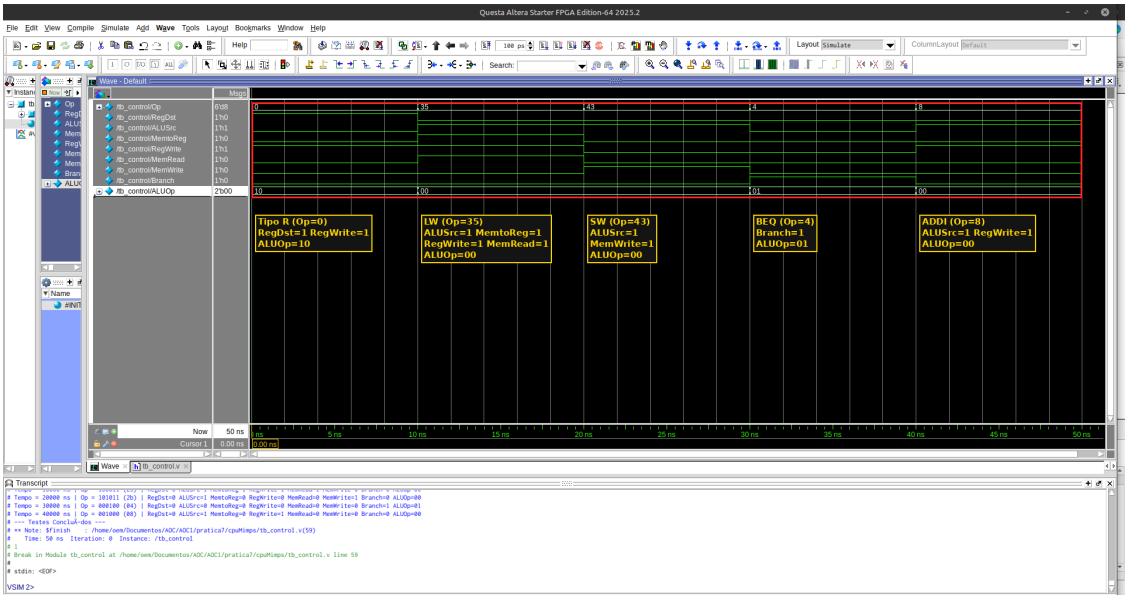


Figura 2 – Forma de onda do controle principal

São aplicados os opcodes do tipo R (0), LW (35), SW (43), BEQ (4) e ADDI (8). O tipo R ativa RegDst e RegWrite com ALUOp=10; o LW ativa ALUSrc, MemtoReg, RegWrite e MemRead; o SW ativa ALUSrc e MemWrite sem escrever no banco; o BEQ ativa apenas Branch com ALUOp=01; e o ADDI ativa ALUSrc e RegWrite com ALUOp=00. Os valores também foram impressos no transcript, coincidindo com a Tabela 1.

3.3 Caminho de dados: ADD (tb_cpuMimps_add.v)

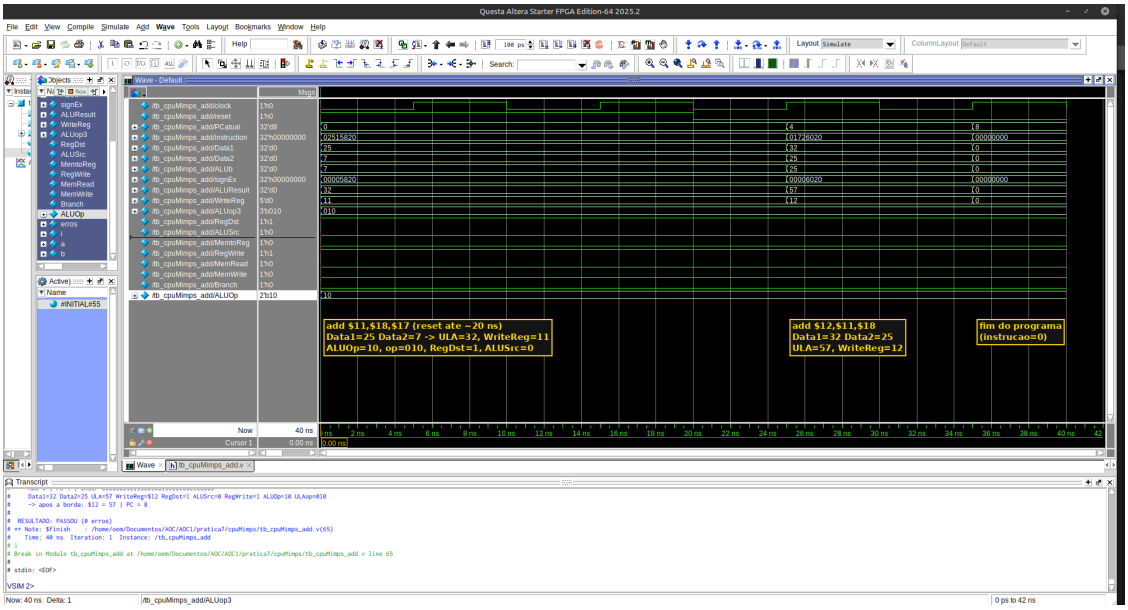


Figura 3 – Forma de onda do caminho de dados executando ADD

Este testbench carrega o programa de instrucoes_add.txt (add $11,$18,$17 e add

$12,$11,$18) com $18=25 e $17=7. Na primeira instrução, Data1=25 e Data2=7 chegam à ULA com ALUOp=10 e operação 010, resultando em 32, gravado em $11 (RegDst=1, ALUSrc=0, RegWrite=1). Na segunda, a leitura de $11 já retorna 32, o que mostra que a escrita da instrução anterior foi concluída, e o resultado 57 é gravado em $12, com o PC avançando de 4 em 4. O transcript mostra "PASSOU (0 erros)".

3.4 Caminho de dados: ADDI, SLT e demais instruções (tb_cpuMimps.v)

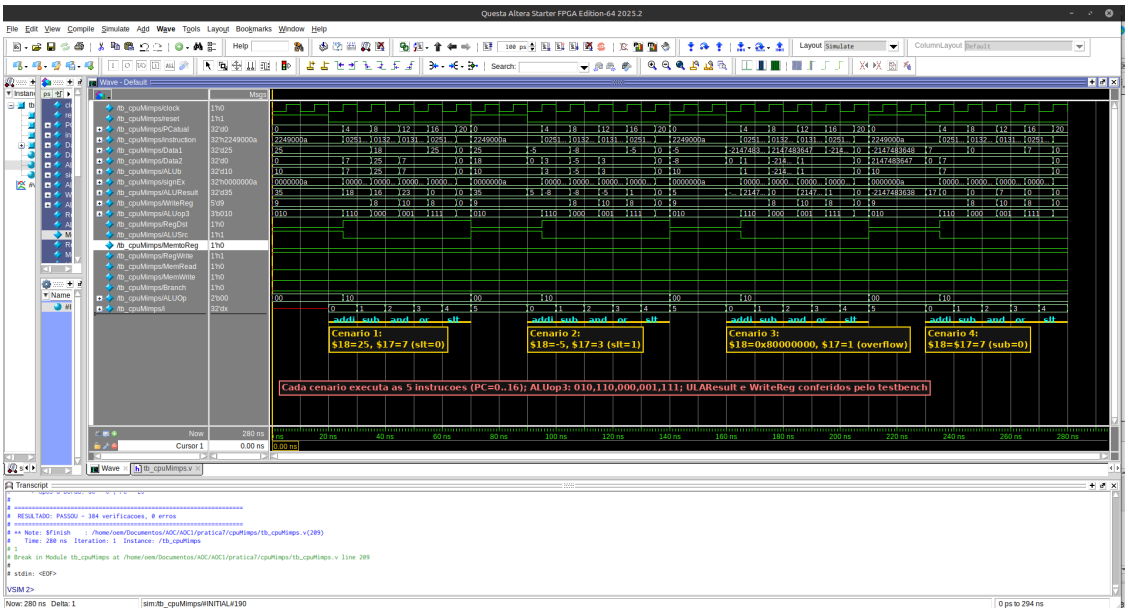


Figura 4 – Forma de onda do caminho de dados executando o programa completo (4 cenários)

Este testbench autoverificável executa as cinco instruções do programa em quatro cenários, conferindo PC, instrução, sinais de controle, ALUOp, operação da ULA, WriteReg, Data1, Data2, ALUb, resultado da ULA, o valor gravado no banco após a borda e o PC+4. Os cenários foram escolhidos para exercitar casos diferentes, principalmente o slt: valores positivos (25 e 7, slt=0), negativos (-5 e 3, slt=1), overflow na subtração (0x80000000 e 1, em que a diferença muda de sinal mas o slt continua correto, igual a 1) e valores iguais (7 e 7, sub=0).

No primeiro cenário, o addi faz ALUb receber o imediato 10 (ALUSrc=1, WriteReg=$9, RegDst=0) e produz 35; as demais instruções usam ALUSrc=0 e RegDst=1, e a operação da ULA segue 110, 000, 001 e 111 para sub, and, or e slt, resultando em 18, 16, 23 e 0. O resultado final no transcript foi "PASSOU – 384 verificações, 0 erros".

4. Conclusão

Os dois blocos de controle foram implementados e validados individualmente, e o caminho de dados completo executou corretamente ADD, ADDI, SUB, AND, OR e SLT em todas as simulações, inclusive em casos de borda como números negativos e overflow. O reaproveitamento do somador da ULA para subtração e slt simplificou o hardware, e o top-level com seleção por chaves permite acompanhar na placa DE10-Lite cada etapa da execução (busca, decodificação, leitura do banco, execução e escrita), e não apenas o resultado final.